CSA0823 - PYTHON PROGRAMMING

ASSIGNMENT – 3 (CO3)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course Code: CSA0823

Submitted by:

Name: D.S.Tharuniya Priya

Register Number: 192511317

Programme: B.Tech. Computer Science and Engineering

Year: II Year

Submitted to:

Faculty Name: Hariramachandran

Department of Computer Science and Engineering

SIMATS Engineering

Academic Year: 2026–2027

QUESTION 1 – LISTS AND SLICING: DAILY RAINFALL RECORDS

1. Aim

To develop a Python program using lists and slicing to analyse daily rainfall records, calculate weekly totals, identify maximum and minimum rainfall, calculate moving averages, and compare the performance of different algorithms.

2. Problem Description

A meteorological department records the daily rainfall of a city for 365 days. The rainfall values are stored in a Python list. The program analyses the data to calculate weekly rainfall totals, identify the wettest and driest days, calculate seven-day moving averages, and count dry days. It also compares a naive moving-average algorithm with an optimized running-sum algorithm.

3. Objectives

- To store daily rainfall measurements in a list.
- To calculate weekly rainfall totals using slicing.
- To identify the maximum and minimum rainfall values and their positions.
- To calculate seven-day moving averages.
- To identify days with zero rainfall.
- To compare the execution time of two moving-average algorithms.

4. Algorithm

1. Generate 365 rainfall readings and store them in a list.
2. Divide the list into groups of seven elements using slicing.
3. Calculate the total rainfall for each week.
4. Find the maximum and minimum rainfall values.
5. Identify the day numbers corresponding to these values.
6. Calculate seven-day moving averages using both naive and running-sum methods.

7. Identify and count dry days.
8. Test empty and shorter-than-seven-element lists.
9. Generate larger datasets and compare the execution times of both methods.

5. List Operations and Slicing

A list is suitable for storing rainfall records because it is ordered, mutable, and supports indexing and slicing. It allows measurements to be accessed by day and modified when corrections are required.

Operation	Description	Average Time Complexity
append()	Adds an element at the end	O(1) amortized
insert()	Adds an element at a specified position	O(n)
pop()	Removes an element by index	O(n) for an arbitrary index; O(1) at the end
sort()	Sorts the elements	O(n log n)

The following slicing expressions have different purposes:

- readings[7:14] selects elements at indices 7 through 13, representing the second group of seven readings.
- readings[-7:] selects the final seven readings.
- readings[::7] selects every seventh element, starting from index 0. It does not calculate weekly totals.

6. Moving Average Algorithms

A seven-day moving average is calculated by adding seven consecutive rainfall values and dividing the total by seven.

Naive method: Each window is calculated independently. It performs six additions for each seven-element window. For n readings, the number of windows is $n - 6$, so the number of additions is:

$6(n - 6)$

Running-sum method: The first window requires six additions. Each subsequent window removes the oldest value and adds the newest value. This requires $n - 7$ further additions and $n - 7$ subtractions. Therefore, the total additions are:

$6 + (n - 7) = n - 1$

The naive approach requires $O(7n)$ time, which is $O(n)$ when the window size is fixed. The running-sum method requires $O(n)$ time and performs fewer arithmetic operations.

7. Pseudocode for Running-Sum Moving Average

1. Start.
2. Read the rainfall list.
3. If the number of readings is less than seven, return an empty result.
4. Calculate the sum of the first seven readings.
5. Divide the sum by seven and store the first average.
6. For each remaining reading:
    - Subtract the oldest reading from the current sum.
    - Add the newest reading.
    - Calculate and store the average.
7. Display the moving averages.
8. Stop.

8. Experimental Investigation

The program generates datasets containing 365, 36,500, and 365,000 readings. It measures the execution time of both moving-average methods using time.perf_counter().

Table 1: Moving Average Execution Time

Number of readings	Naive execution time	Running-sum execution time
365	0.000092794	0.000032832

36,500	0.009330441	0.003899775
365,000	0.131179974	0.038644614

Table 2: Rainfall Analysis

Parameter	**Result**
Total daily readings	365
Number of weekly totals	53
Number of moving averages	359
Maximum rainfall	50 mm
Minimum rainfall	0 mm
Number of dry days	7
Moving average results match	Yes

Table 3: Wettest and Driest Days

Category	**Day numbers**
Maximum rainfall days	154, 283, 340, 341, 347
Minimum rainfall days	36, 166, 182, 204, 260, 324, 337

The running-sum method is expected to perform better because it avoids recalculating the sum of every seven-element window.

9. Edge Cases

- Empty list: The program returns an empty moving-average list.
- Fewer than seven readings: No complete seven-day window exists, so the result is empty.
- Zero rainfall: A reading of zero is counted as a dry day.

- Final partial week: The remaining readings are included in the final weekly total.

10. Late Insertion and Alternative Data Structures

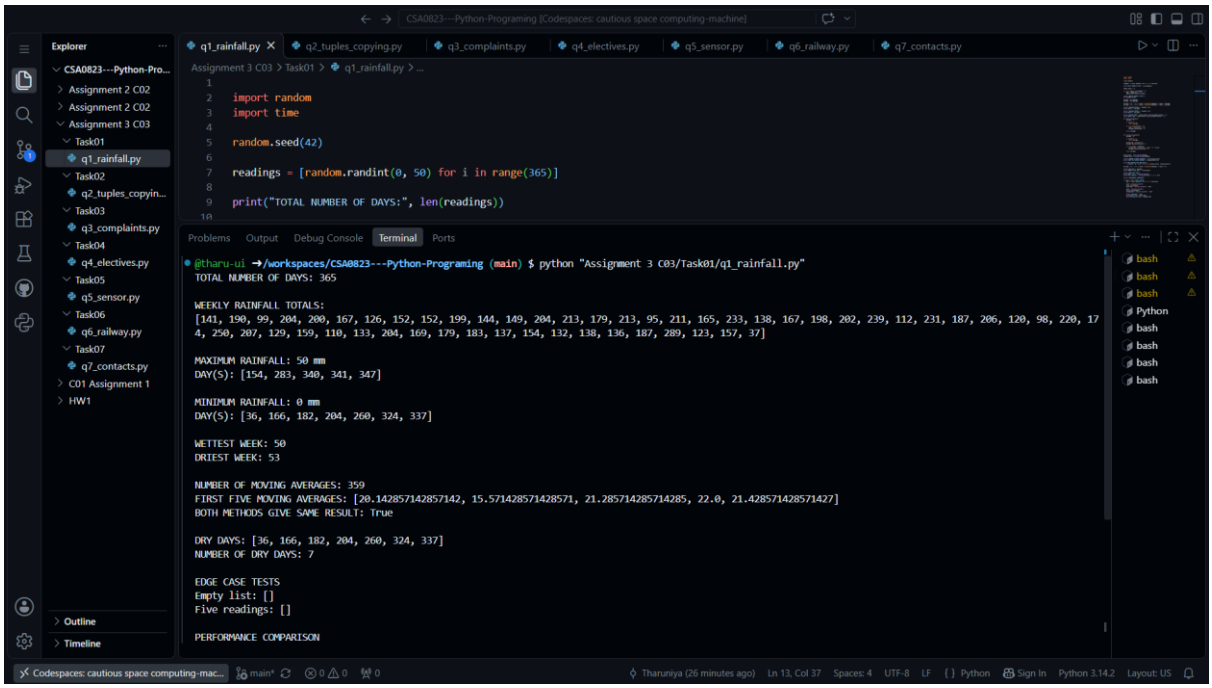
Inserting a new reading in the middle of a Python list requires shifting the elements after the insertion position. Therefore, the operation takes O(n) time in the worst case.

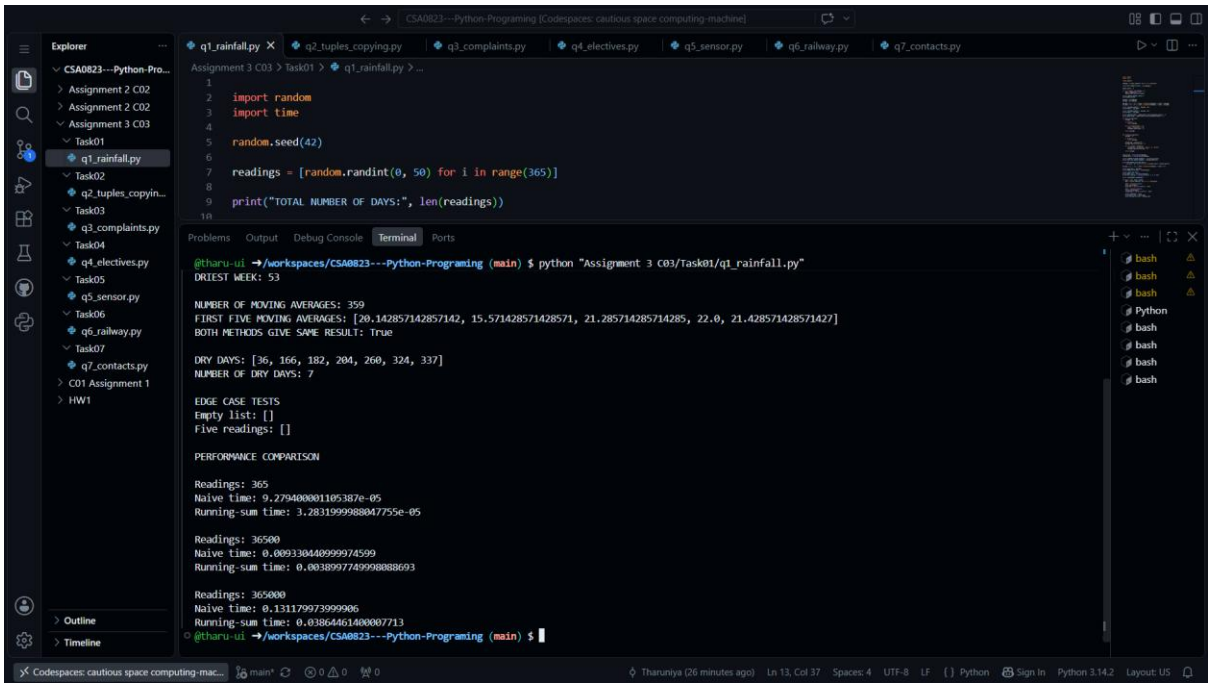
A collections.deque with maxlen=7 can be used to maintain a sliding window efficiently. Appending and removing elements at either end are O(1) operations. A dictionary keyed by date can also be used when records need to be retrieved by date.

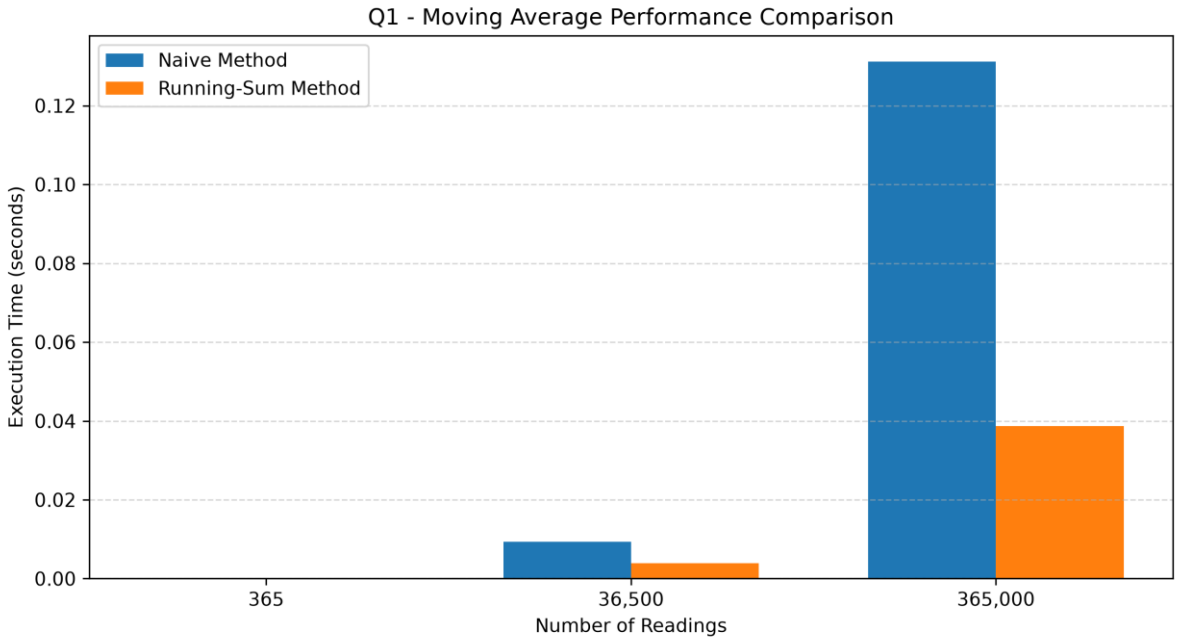
Approach	Time Complexity	Additional Space
Naive moving average	O(7n), equivalent to O(n) for fixed window	O(n) for stored averages
Running-sum moving average	O(n)	O(n) for stored averages
Deque-based sliding window	O(n) overall	O(7) for the window

11. Result

The rainfall analysis program was developed using Python lists and slicing. It calculates weekly totals, maximum and minimum rainfall, seven-day moving averages, and dry days. The performance of the naive and running-sum algorithms is compared using different dataset sizes.




q1_rainfall.py
q2_tuples_copying.py
q3_complaints.py
q4_electives.py
q5_sensor.py
q6_railway.py
q7_contacts.py
import random
import time
random.seed(42)
readings = [random.randint(0, 50) for i in range(365)]
print("TOTAL NUMBER OF DAYS:", len(readings))
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ python "Assignment 3 C03/Task01/q1_rainfall.py"
DRIEST WEEK: 53
NUMBER OF MOVING AVERAGES: 359
FIRST FIVE MOVING AVERAGES: [20.142857142857142, 15.571428571428571, 21.285714285714285, 22.0, 21.428571428571427]
BOTH METHODS GIVE SAME RESULT: True
DRY DAYS: [36, 166, 182, 204, 260, 324, 337]
NUMBER OF DRY DAYS: 7
EDGE CASE TESTS
Empty list: []
Five readings: []
PERFORMANCE COMPARISON
Readings: 365
Naive time: 9.279400001105387e-05
Running-sum time: 3.2831999988047755e-05
Readings: 36500
Naive time: 0.009330440999974599
Running-sum time: 0.0038997749998088693
Readings: 365000
Naive time: 0.131179973999906
Running-sum time: 0.038644614000771 3
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $


Q1 - Moving Average Performance Comparison
Naive Method
Running-Sum Method
Execution Time (seconds)
0.00
0.02
0.04
0.06
0.08
0.10
0.12
365
36,500
365,000
Number of Readings

QUESTION 2 – TUPLES, MUTABILITY AND COPYING

1. Aim

To study Python tuples, mutability, aliasing, shallow copying, and deep copying, and to understand their applications in storing coordinates and multidimensional data.

2. Problem Description

A geographical information system stores coordinates as pairs of values. Tuples are used to represent these coordinates because they are immutable and can be used as dictionary keys. The program also demonstrates the difference between aliasing, shallow copying, and deep copying through nested lists.

3. Objectives

- To create and access tuples.
- To use tuples as dictionary keys.
- To understand tuple unpacking.
- To demonstrate aliasing and copying.
- To create independent two-dimensional lists.
- To compare list and tuple memory usage.

4. Algorithm

1. Create coordinate tuples and use them as dictionary keys.
2. Access coordinate values using indexing.
3. Demonstrate tuple unpacking in a loop.
4. Create a nested list using repeated references.
5. Modify one row and observe the effect of aliasing.
6. Create an independent grid using list comprehension.
7. Demonstrate shallow copying and deep copying.
8. Display object identities using id() and compare references using is.

9. Measure the memory occupied by lists and tuples.
10. Test the behaviour of mutable objects stored inside tuples.

5. Tuples and Dictionary Keys

A tuple is an ordered and immutable collection. Once created, its elements cannot be reassigned. A tuple containing only hashable elements can be used as a dictionary key.

Example:

location = (13.0827, 80.2707)

A dictionary can store information using the coordinate tuple as a key:

places = {(13.0827, 80.2707): "Chennai"}

A list cannot be used as a dictionary key because lists are mutable and unhashable. Tuples containing mutable elements, such as lists, are also unsuitable as dictionary keys.

6. Tuple Unpacking

Tuple unpacking assigns tuple elements to separate variables.

Example:

for name, city in records:

Here, each tuple in the collection is unpacked into the variables name and city.

7. Aliasing, Shallow Copy and Deep Copy

Aliasing: Two variables refer to the same object. A modification through one variable is visible through the other.

Shallow copy: Creates a new outer object but retains references to the nested objects. Changes to nested mutable objects may therefore affect both copies.

Deep copy: Creates a new object and recursively copies nested objects. Changes to the copied nested objects do not affect the original.

The expression [[0] * 3] * 3 creates three references to the same inner list. Changing one element changes the corresponding element in every row.

The correct method is:

grid = [[0 for j in range(3)] for i in range(3)]

This creates a separate inner list for every row.

8. Complete the Code – Independent Grid

The required 3 × 4 grid is created using nested list comprehension. Only the first row is modified.

Expected output:

bad : [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]]

good: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]

True False

T. Nagar

The first two results demonstrate shared references in the incorrectly created grid and independent rows in the correctly created grid.

9. Memory Comparison

The sys.getsizeof() function measures the shallow memory size of an object. The program compares lists and tuples containing one million integer references.

The exact memory values depend on the Python implementation and system. sys.getsizeof() does not recursively include the memory occupied by all referenced objects.

10. Alternative Approach

A sparse matrix can be represented using a dictionary with coordinate tuples as keys.

Example:

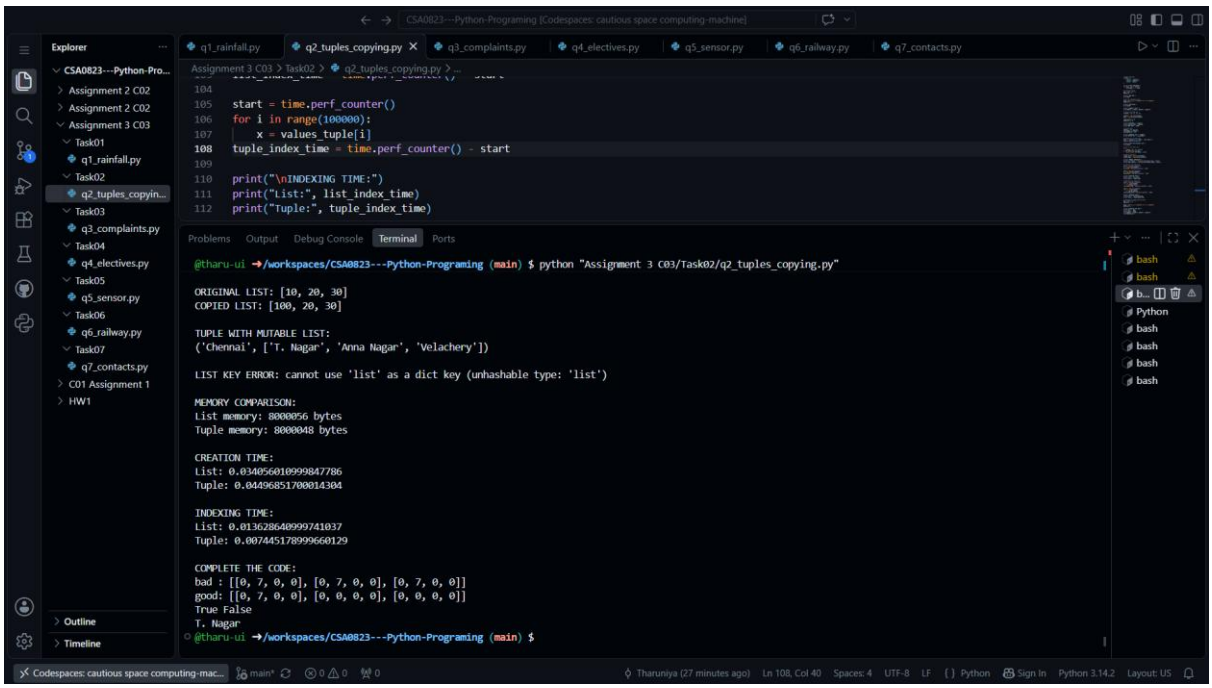
matrix = {(0, 1): 7, (2, 3): 5}

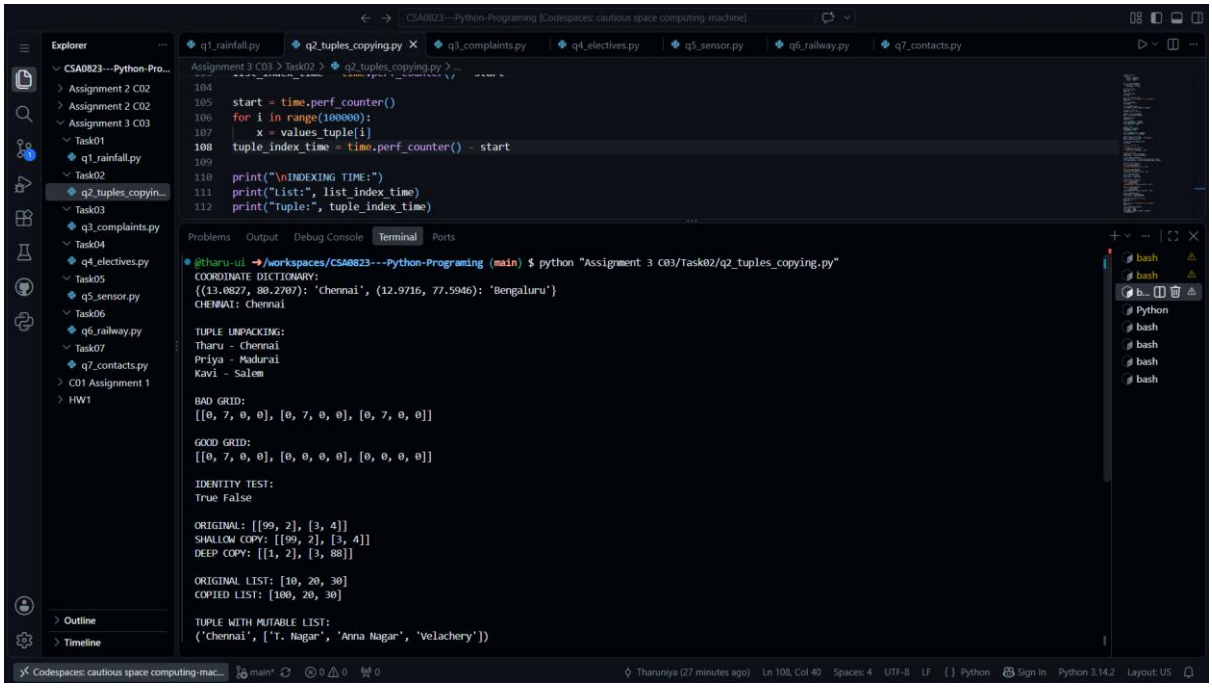
This approach stores only non-zero elements and can save memory when most matrix positions contain zero.

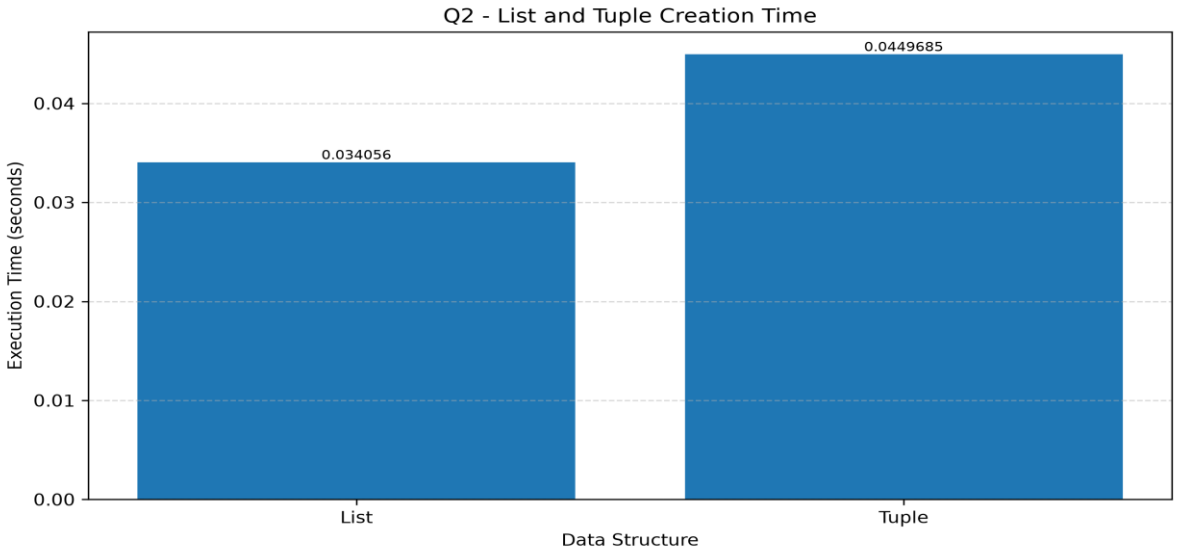
11. Result

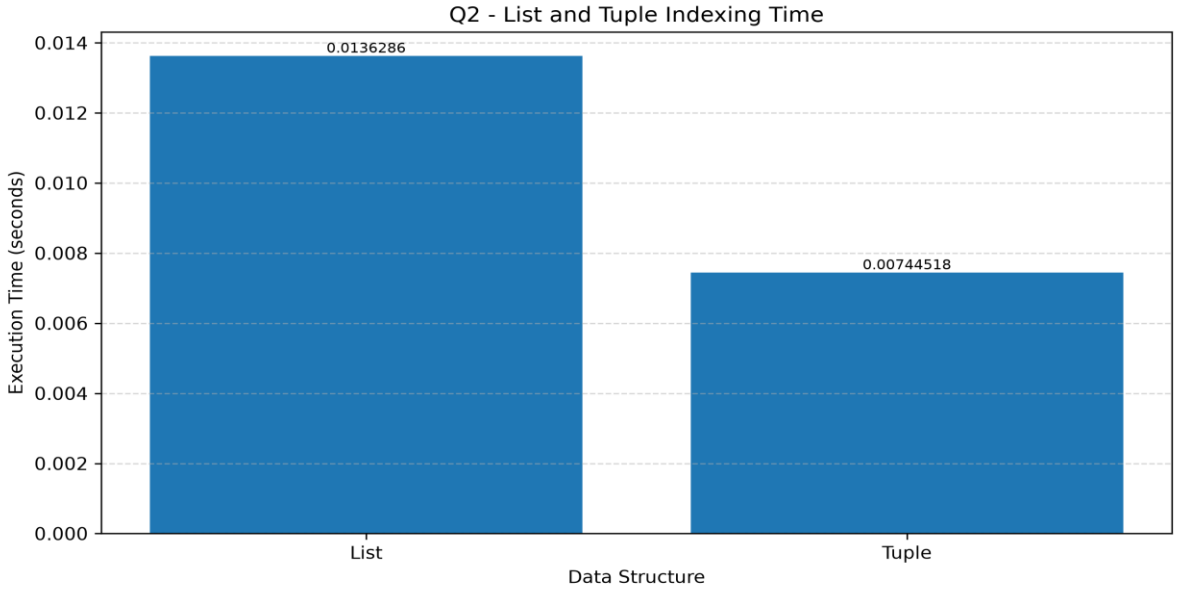
The program demonstrates tuple usage, tuple unpacking, dictionary keys, aliasing, shallow copying, deep copying, and independent nested lists. It also compares list and tuple memory usage.

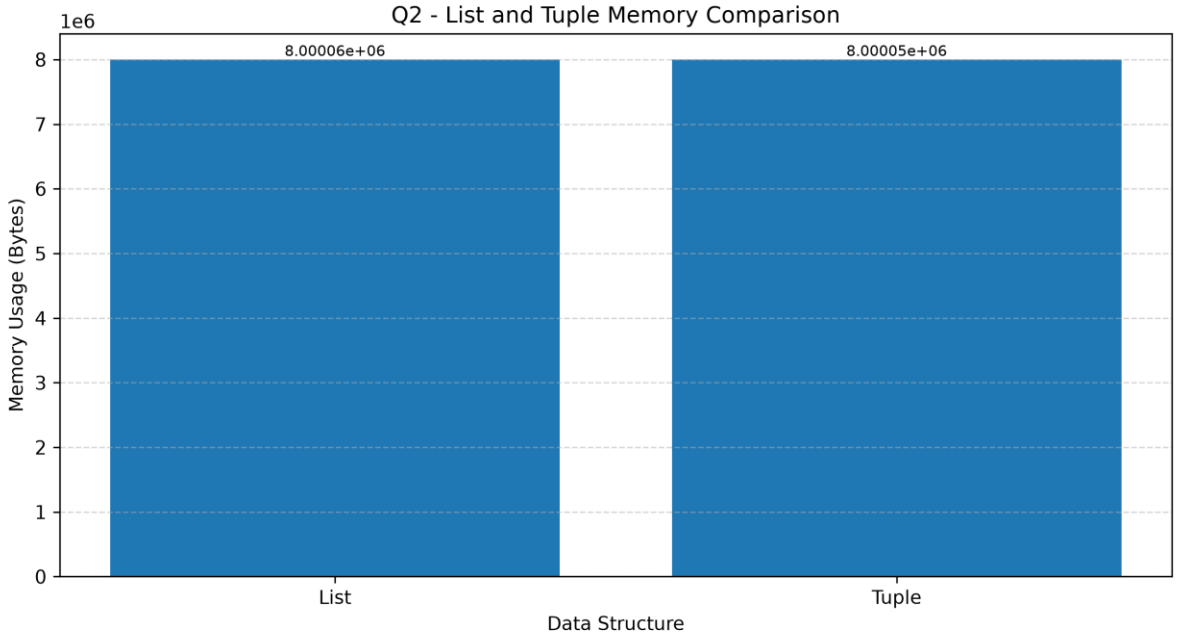
Measurement	**List**	**Tuple**
Memory usage (bytes)	8,000,056	8,000,048
Creation time (seconds)	0.034056011	0.044968517
Indexing time (seconds)	0.013628641	0.007445179






Q2 - List and Tuple Creation Time
0.0449685
0.034056
0.04
0.03
0.02
0.01
0.00
Execution Time (seconds)
List
Tuple
Data Structure


Q2 - List and Tuple Indexing Time
0.0136286
0.00744518
0.014
0.012
0.010
0.008
0.006
0.004
0.002
0.000
Execution Time (seconds)
List
Tuple
Data Structure


Q2 - List and Tuple Memory Comparison
1e6
8.00006e+06
8.00005e+06
8
7
6
5
4
3
2
1
0
Memory Usage (Bytes)
List
Tuple
Data Structure

QUESTION 3 – DICTIONARIES: FARMER HELPLINE COMPLAINT ANALYSIS

1. Aim

To develop a Python program using dictionaries and sets to analyse farmer helpline complaints, count frequently occurring words, group complaints by district, and identify the most common complaint terms.

2. Problem Description

A farmer helpline receives complaints from different districts. The complaints contain repeated words and common stop words. The program processes the complaint text, counts word frequencies, groups complaints by district, and displays the most frequent words.

3. Objectives

- To count word occurrences using dictionaries.
- To use get() and setdefault().
- To group complaints by district.
- To remove stop words using a set.
- To identify the top k frequent words.
- To compare dictionary-based counting with list-based counting.

4. Algorithm

1. Read the complaint records.
2. Convert each complaint to lowercase.
3. Remove punctuation and split the complaint into words.
4. Remove common stop words using a set.
5. Count each word using a dictionary.
6. Group complaints using the district name as the dictionary key.
7. Sort the word frequencies by descending count and alphabetical order.
8. Display the top k words.

9. Generate larger datasets and compare execution times.

5. Dictionary Operations

A dictionary stores data as key-value pairs. Each word is used as a key and its frequency is stored as the corresponding value.

The get() method retrieves a value without raising an error when the key is absent.

Example:

count[word] = count.get(word, 0) + 1

The setdefault() method returns the existing value or inserts the key with a default value.

Dictionaries preserve insertion order in Python 3.7 and later.

Method	**Purpose**
keys()	Returns dictionary keys
values()	Returns dictionary values
items()	Returns key-value pairs
get()	Retrieves a value with a default
setdefault()	Retrieves or inserts a key

6. Hash-Based Lookup

Python dictionaries use hashing to locate keys. Average lookup, insertion, and update operations take O(1) time. In contrast, using list.count() separately for every distinct word requires repeated scanning of the complaint data.

If there are n total words and d distinct words, repeated list counting can take O(n × d) time, while dictionary-based counting takes O(n) average time.

7. Complete the Code – Complaint Word Frequency

Input:

calls = [

```
    "seeds not received seeds delayed",
    "fertiliser price high",
    "seeds price high subsidy not received",
]
STOP = {"not", "high"}
```

Expected output:

```
[('seeds', 3), ('price', 2), ('received', 2)]
```

The words price and received both have a frequency of two. Alphabetical ordering places price before received.

8. Experimental Investigation

Generate datasets containing 1,000, 10,000, and 100,000 complaint words. Measure the execution time of dictionary counting, repeated list.count(), and collections.Counter.

Dataset size	Dictionary time	list.count() time	Counter time
1,000	0.000132558	0.000057116	0.000125695
10,000	0.001377553	0.000415767	0.001281093
100,000	0.035158101	0.016216515	0.044500674

The dictionary and Counter approaches are expected to scale better because they count words in a single traversal.

9. Edge Cases

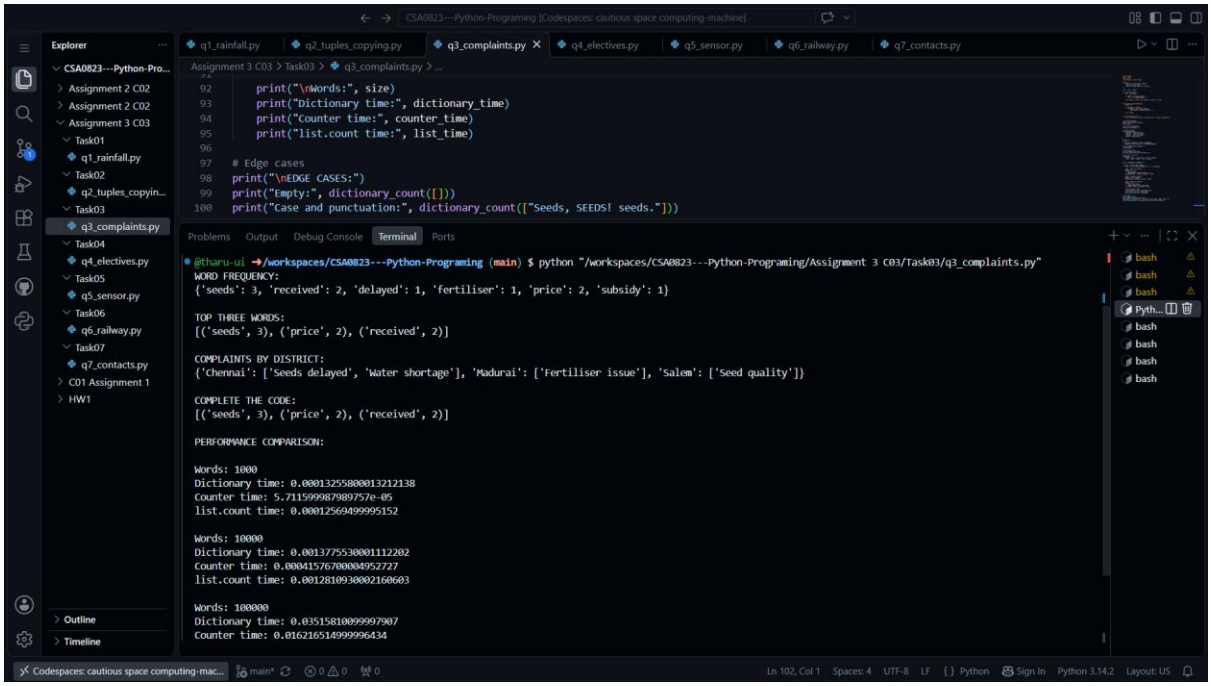
- Empty complaint list.
- Words with different letter cases.
- Punctuation attached to words.
- Complaints containing only stop words.
- Repeated words in the same complaint.

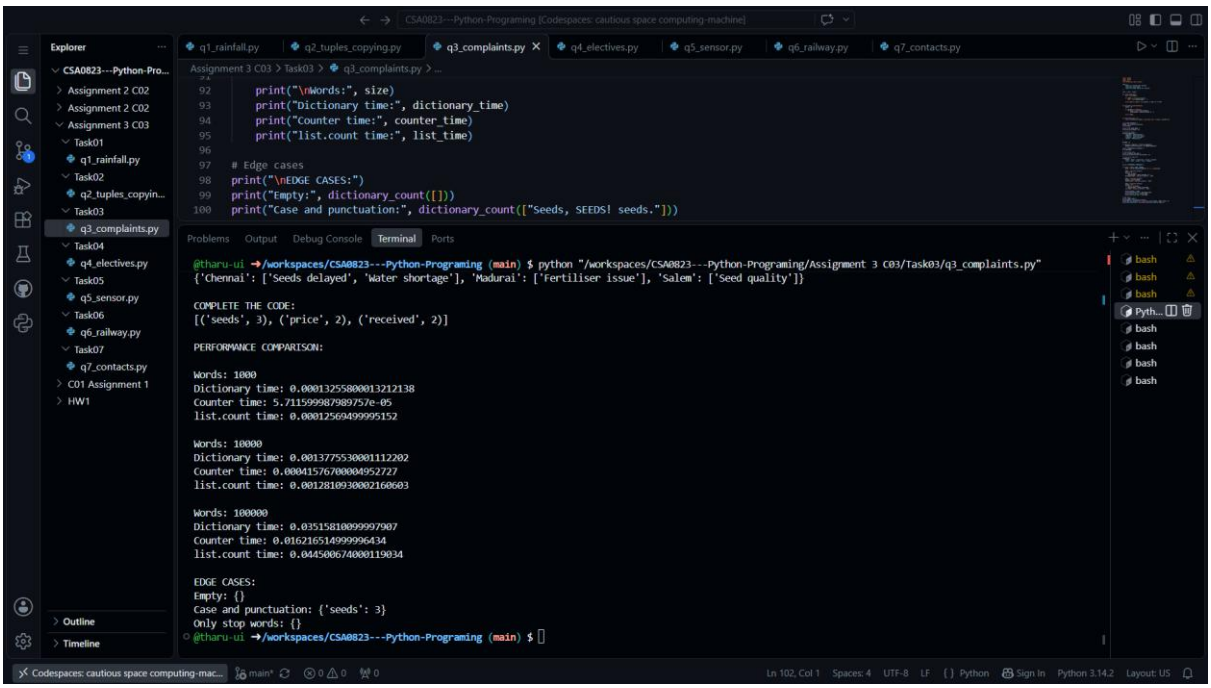
10. Alternative Approach

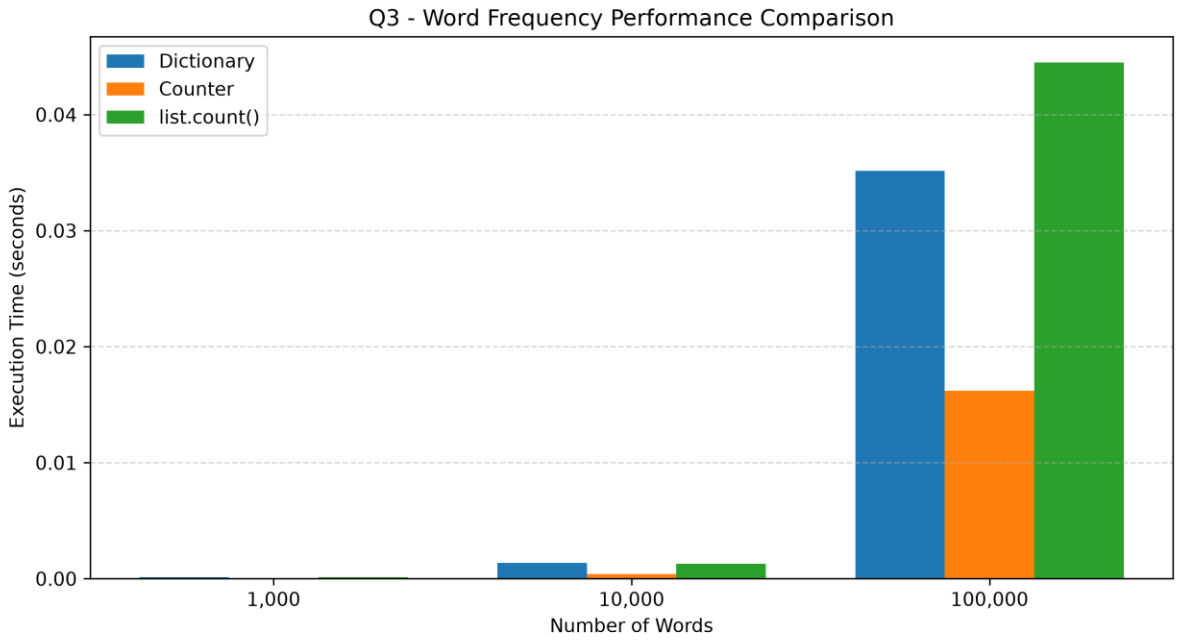
The collections.Counter class provides a convenient alternative for counting word frequencies. It is based on dictionary-like counting and provides methods such as most_common().

11. Result

The complaint analysis program uses dictionaries and sets to count words, group complaints, remove stop words, and identify frequent complaint terms. Its performance is compared with list-based counting and Counter.






Q3 - Word Frequency Performance Comparison
Dictionary
Counter
list.count()
Execution Time (seconds)
0.00
0.01
0.02
0.03
0.04
1,000
10,000
100,000
Number of Words

QUESTION 4 – SETS: ELECTIVE REGISTRATION CLASH DETECTION

1. Aim

To develop a Python program using sets to identify students registered for two electives, students registered for exactly one elective, and students who have not registered for either elective.

2. Problem Description

A college offers Artificial Intelligence and Internet of Things electives. The registration lists may contain repeated student IDs. Sets are used to remove duplicates and perform efficient membership and comparison operations.

3. Objectives

- To create and manipulate sets.
- To remove duplicate student IDs.
- To calculate union and intersection.
- To calculate difference and symmetric difference.
- To identify students who have not registered for either elective.
- To compare set operations with nested loops.

4. Algorithm

1. Create two sets containing AI and IoT registrations.
2. Create a set containing all eligible student IDs.
3. Calculate the intersection to identify students registered for both electives.
4. Calculate the symmetric difference to identify students registered for exactly one elective.
5. Calculate the difference between all students and the union of both elective sets.
6. Sort the results before displaying them.
7. Compare set-based operations with alternative approaches.

5. Set Operations

Operation	Symbol	Meaning
Union	`A U B`	Elements in A or B
Intersection	A & B	Elements common to A and B
Difference	A - B	Elements in A but not B
Symmetric difference	A ^ B	Elements in exactly one set

Sets automatically remove duplicate elements. They provide average O(1) membership testing and do not preserve a guaranteed display order.

A set can contain immutable tuples, but it cannot contain lists because lists are mutable and unhashable.

6. Complete the Code – Elective Clash Detection

Input:

ai_elective = {"192311001", "192311004", "192311007", "192311009"}

iot_elective = {"192311002", "192311004", "192311009", "192311010"}

all_students = {f"1923110{i:02d}" for i in range(1, 11)}

Expected output:

clash : ['192311004', '192311009']

exactly one: ['192311001', '192311002', '192311007', '192311010']

neither : ['192311003', '192311005', '192311006', '192311008']

The intersection identifies the students registered for both electives. The symmetric difference identifies students registered for exactly one elective. The remaining students are found by subtracting the union from the complete student set.

7. Performance Analysis

Membership testing in a set takes O(1) average time, while membership testing in a list takes O(n) time. Comparing every element of two lists using nested loops can take O(n × m) time.

Approach	Expected Time Complexity
Nested loops	O(n × m)
Set intersection	O(n + m) average
Sorting-based comparison	O(n log n + m log m)

8. Experimental Investigation

Measure membership testing for datasets containing 10,000, 100,000, and 1,000,000 student IDs. Compare list membership, set membership, nested-loop comparison, and set intersection where applicable.

Test empty sets, identical sets, and a small manually verified dataset.

9. Alternative Approach

Lists can be used to store registrations, but duplicates must be removed manually. Sorting and comparing lists is another alternative, although set operations are generally simpler and more efficient for membership and clash detection.

10. Result

The elective registration program uses set operations to detect clashes, identify students registered for exactly one elective, and find students who have not registered for either elective.

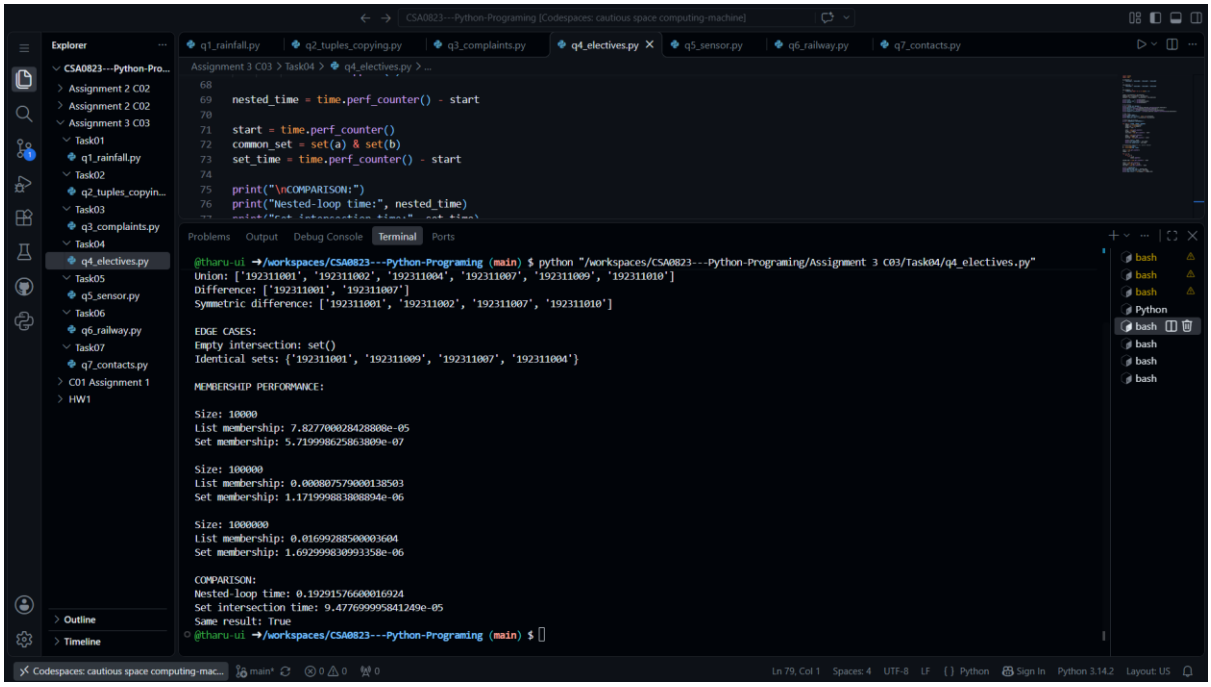
Table: Membership Testing Performance

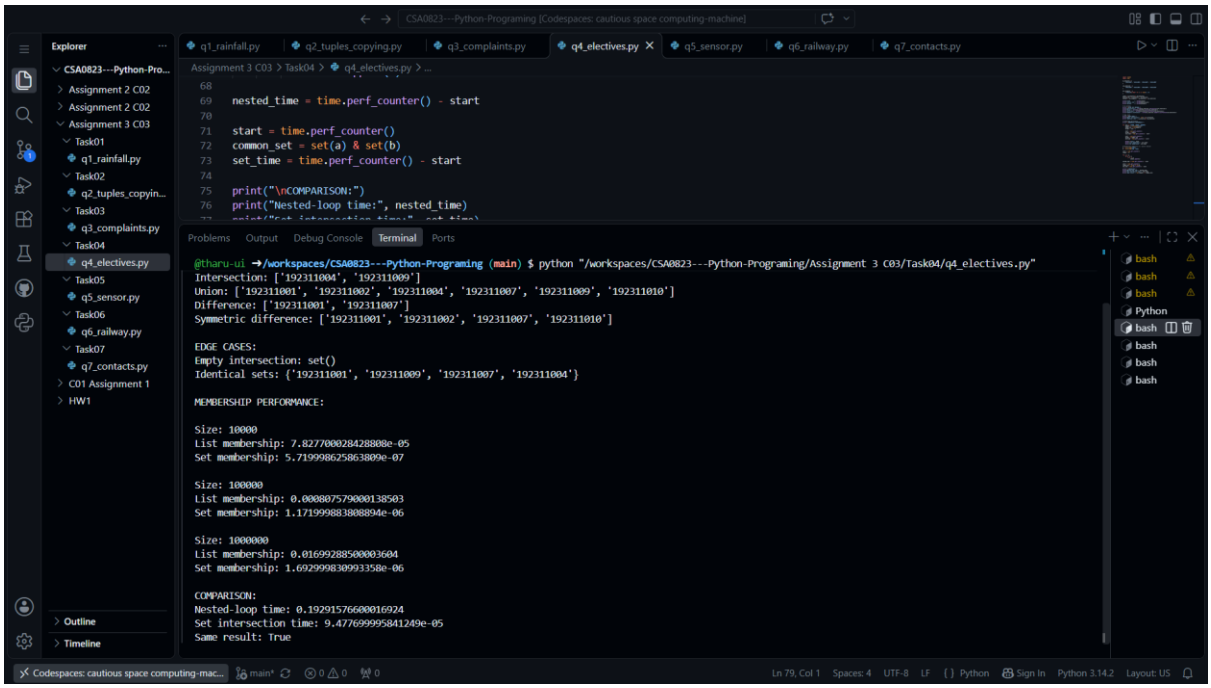
Dataset size	List membership (seconds)	Set membership (seconds)
10,000	0.000078277	0.000000572
100,000	0.000807580	0.000001172
1,000,000	0.016992885	0.000001693

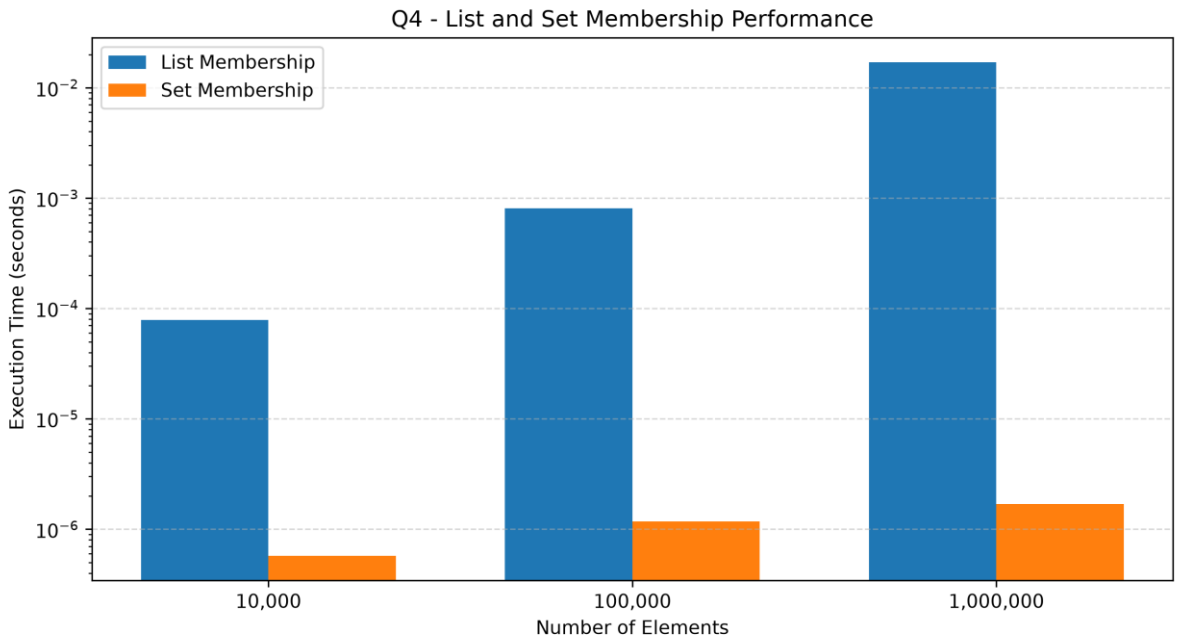
Table: Intersection Performance

Method	Execution time (seconds)
Nested loop	0.192915766
Set intersection	0.000094777

Both methods produced the same intersection result.






Q4 - List and Set Membership Performance
List Membership
Set Membership
Execution Time (seconds)
10^{-2}
10^{-3}
10^{-4}
10^{-5}
10^{-6}
10,000
100,000
1,000,000
Number of Elements

QUESTION 5 – LIST COMPREHENSION: CLEANING SENSOR READINGS

1. Aim

To develop a Python program using list comprehension, map(), filter(), and enumerate() to clean sensor readings, convert temperatures, and identify alarm conditions.

2. Problem Description

A temperature monitoring system receives sensor readings in Celsius. The valid operating range is from −50°C to 50°C. The program removes invalid readings, converts valid values into Fahrenheit, and identifies readings outside the normal operating range of 2°C to 8°C.

3. Objectives

- To validate sensor readings.
- To clean invalid data using list comprehension.
- To convert Celsius into Fahrenheit.
- To identify alarm conditions.
- To use map(), filter(), and enumerate().
- To compare list comprehension with generator expressions.

4. Algorithm

1. Read the sensor readings.
2. Retain only values between −50°C and 50°C.
3. Convert the cleaned readings into Fahrenheit.
4. Identify values below 2°C or above 8°C.
5. Use enumerate() to display the position of each alarm reading.
6. Compare the results generated by different techniques.
7. Measure execution time and memory usage for larger datasets.

5. List Comprehension

List comprehension provides a compact method of creating lists.

General syntax:

[expression for item in iterable if condition]

It can be used to filter invalid sensor values and transform valid readings.

Example:

cleaned = [x for x in readings if -50 <= x <= 50]

A dictionary comprehension creates dictionaries, while a set comprehension creates sets.

6. Temperature Conversion

The formula for converting Celsius into Fahrenheit is:

$F = (C \times 9/5) + 32$

The program applies this formula to every valid reading.

7. Alarm Detection

A temperature alarm is generated when a valid Celsius reading is outside the normal range of 2°C to 8°C.

alarm = [x for x in cleaned if x < 2 or x > 8]

The enumerate() function provides both the index and value while iterating through the readings.

8. map(), filter() and Lambda

- map() applies a function to every element.
- filter() selects elements that satisfy a condition.
- lambda creates a small anonymous function.
- enumerate() returns an index and its corresponding value.

These methods can perform similar operations to list comprehension. List comprehension is often easier to read when filtering and transforming data together.

9. List Comprehension vs Generator Expression

A list comprehension creates and stores all results in memory. A generator expression produces values one at a time and is more memory-efficient when the complete result is not needed immediately.

Method	Time Complexity	Result Storage
List comprehension	O(n)	O(n)
Generator expression	O(n) when fully consumed	O(1) generator state, excluding consumed or stored results
map()	O(n) when fully consumed	Iterator-based
filter()	O(n) when fully consumed	Iterator-based

10. Experimental Investigation

Generate datasets containing 10,000, 100,000, and 1,000,000 readings. Compare execution times of list comprehension and map/filter. Measure memory usage of a list and a generator using sys.getsizeof().

Test two special cases:

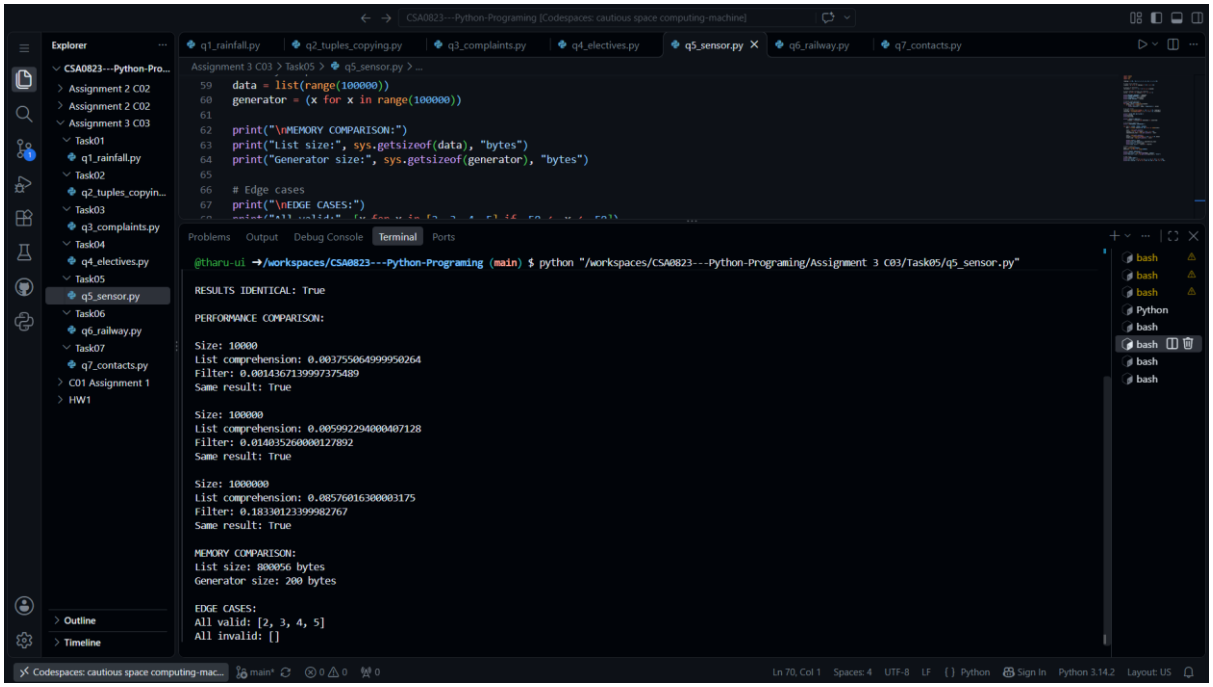
- All readings are valid.
- All readings are invalid.

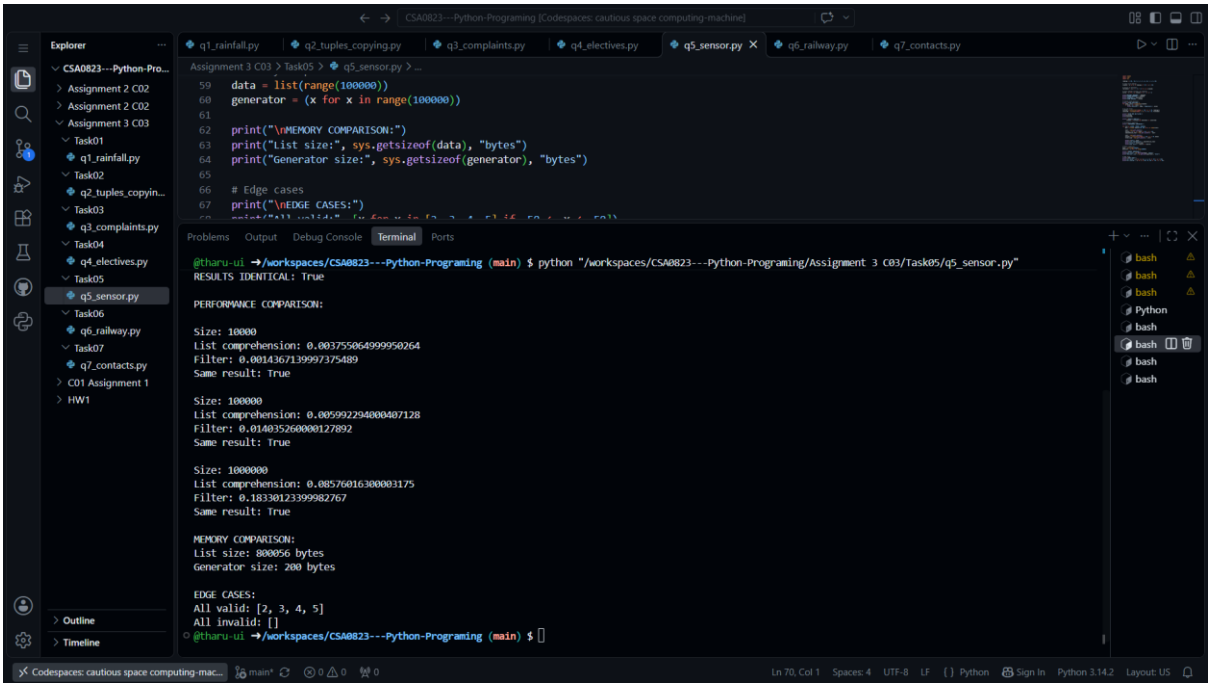
Dataset size	List comprehension time	map/filter time
10,000	0.003755065	0.001436713
100,000	0.005992294	0.014035260
1,000,000	0.085760163	0.183301233

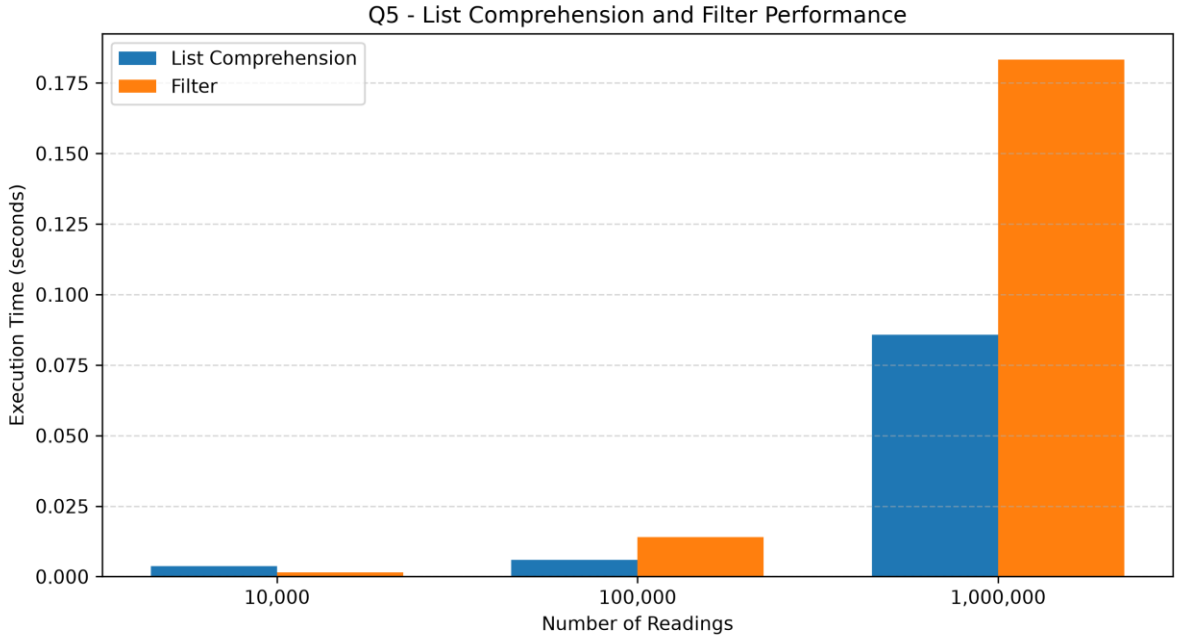
The generator uses less memory because it does not store the entire result in advance.

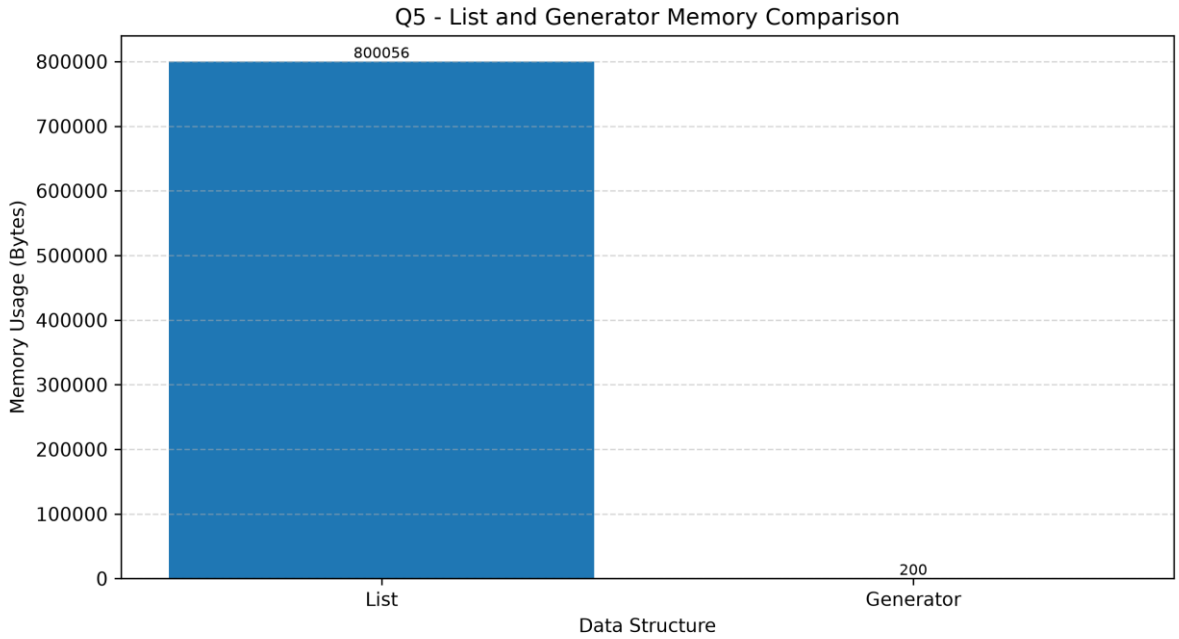
11. Result

The sensor cleaning program validates readings, converts Celsius to Fahrenheit, detects alarm conditions, and demonstrates list comprehension and functional programming techniques.






Q5 - List Comprehension and Filter Performance
List Comprehension
Filter
Execution Time (seconds)
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
10,000
100,000
1,000,000
Number of Readings


Q5 - List and Generator Memory Comparison
800056
200
Memory Usage (Bytes)
800000
700000
600000
500000
400000
300000
200000
100000
0
List
Generator
Data Structure

QUESTION 6 – NESTED DATA: A RAILWAY TIMETABLE

1. Aim

To develop a Python program using nested dictionaries, lists, and tuples to store railway timetable information and retrieve trains based on station and running day.

2. Problem Description

A railway timetable contains information about trains, their station stops, arrival and departure times, and operating days. The program stores this information using nested data structures and provides search and sorting operations.

3. Objectives

- To store train information using nested dictionaries.
- To represent station timings using tuples.
- To store running days using sets.
- To search for trains stopping at a particular station.
- To filter trains based on operating days.
- To sort trains according to departure time.
- To examine the performance of indexed and non-indexed searches.

4. Data Structure Design

The main dictionary uses the train number as its key. Each train number maps to another dictionary containing:

- Train name.
- A list of station tuples.
- A set of running days.

Each station tuple contains the station name, arrival time, and departure time.

Example structure:

trains = {

```
    "12601": {
        "name": "Express",
        "stops": [
            ("Chennai", "08:00", "08:10"),
            ("Katpadi", "10:00", "10:05")
        ],
        "days": {"Monday", "Wednesday", "Friday"}
    }
}
```

5. Algorithm

1. Create a dictionary containing at least five trains.
2. Store each train's name, station list, and operating days.
3. Read the required station and day.
4. Search each train for the requested station.
5. Check whether the train operates on the requested day.
6. Collect matching trains.
7. Sort the matching trains by departure time.
8. Display the train number, name, station, arrival, and departure time.
9. Repeat the search using a station index.
10. Compare query execution time and additional memory.

6. Accessing Nested Data

The third station departure time can be accessed using:

trains["12601"]["stops"][2][2]

The first dictionary lookup identifies the train. The second lookup accesses its station list. The index 2 selects the third station, and the final index 2 selects its departure time.

7. Search Complexity

Without an index, the program checks the station list of every train. If there are T trains and S average stops per train, a query takes O(T × S) time.

A station index maps each station to the trains stopping there. This reduces the search work to the number of trains associated with that station, after the index has been built.

8. Experimental Investigation

Generate timetables containing 100, 1,000, and 10,000 trains. Compare search times with and without a station index.

Number of trains	Without index (seconds)	With index (seconds)
100	0.000018454	0.000009157
1,000	0.000144900	0.000080310
10,000	0.001514179	0.000966466

Test an unknown station and a day on which no matching train operates.

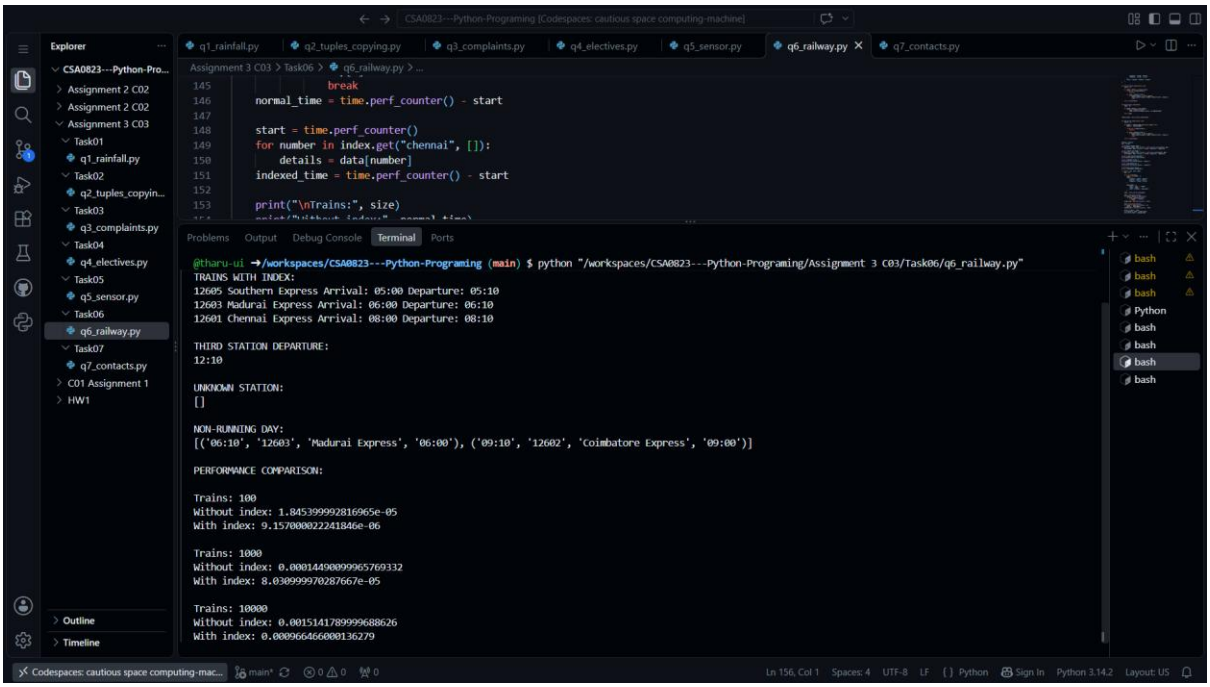
The indexed search was faster than the non-indexed search for all three tested dataset sizes. However, the experiment measures query execution time and does not include the time required to construct the station index.

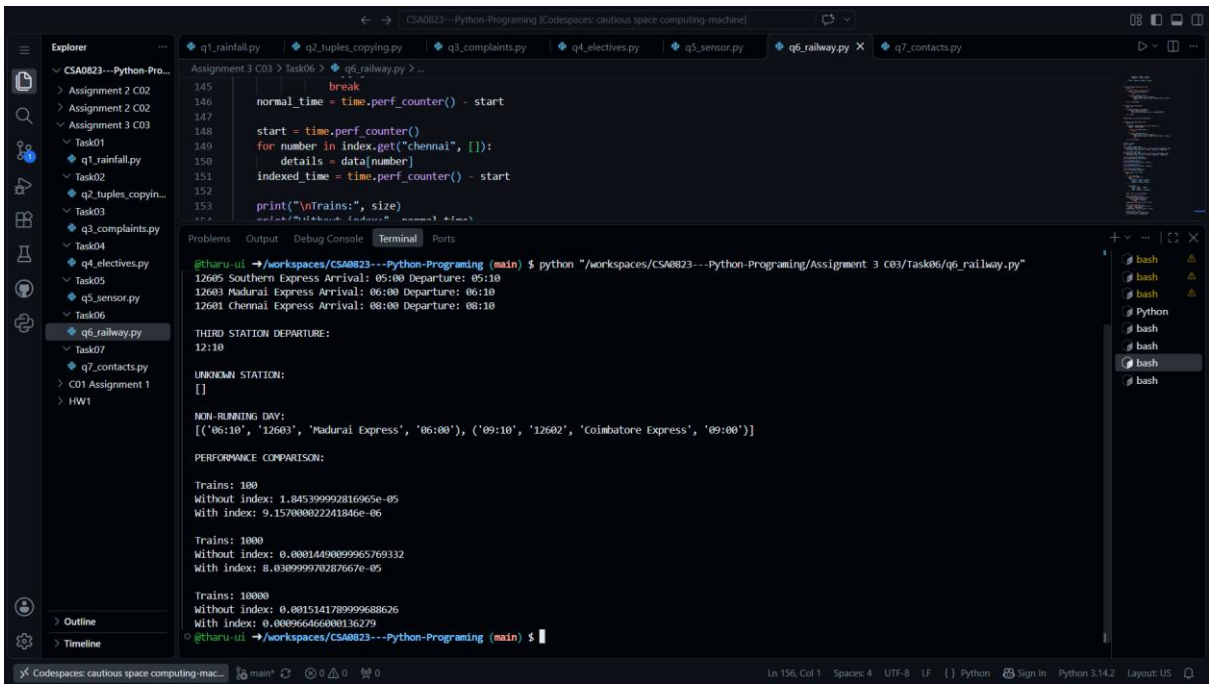
9. Alternative Approach

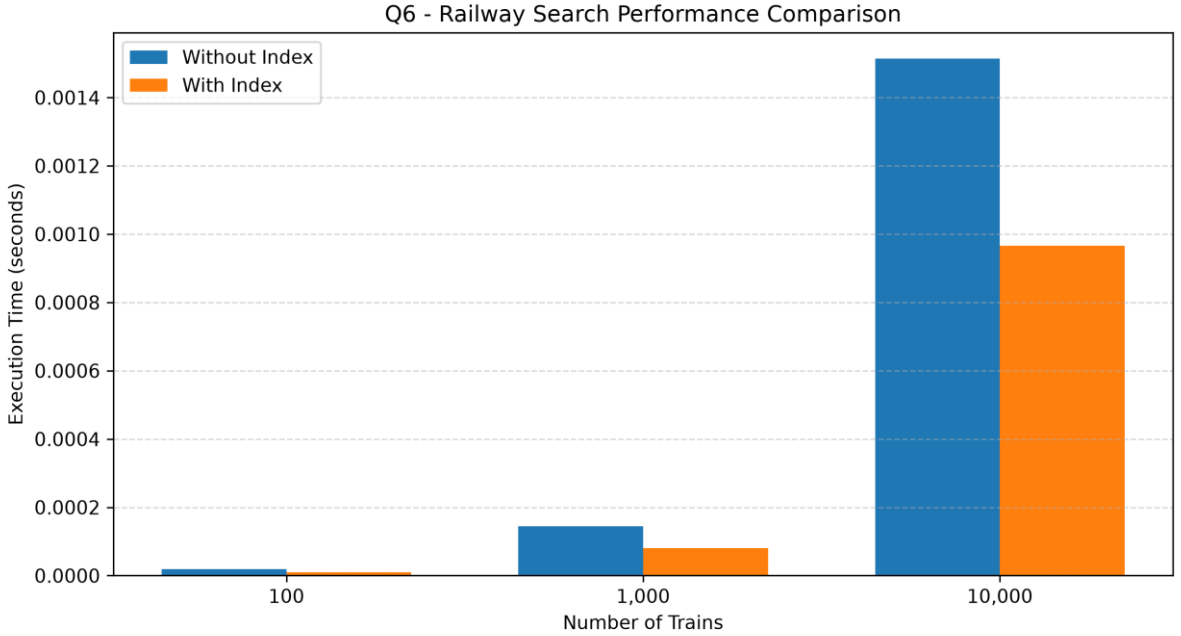
A flat list of tuples can store each train-stop combination. It is simple to scan but requires examining many records for each query. A station index provides faster repeated searches at the cost of additional memory and index-building time.

10. Result

The railway timetable program uses nested dictionaries, lists, tuples, and sets to store train details and retrieve matching services. It also compares indexed and non-indexed search methods.


Explorer
CSA0823---Python-Pro...
Assignment 2 C02
Assignment 2 C02
Assignment 3 C03
Task01
q1_rainfall.py
Task02
q2_tuples_copyin...
Task03
q3_complaints.py
Task04
q4_electives.py
Task05
q5_sensor.py
Task06
q6_railway.py
Task07
q7_contacts.py
C01 Assignment 1
HW1
Outline
Timeline
q1_rainfall.py
q2_tuples_copying.py
q3_complaints.py
q4_electives.py
q5_sensor.py
q6_railway.py
q7_contacts.py
Assignment 3 C03 > Task06 > q6_railway.py > ...
145 break
146 normal_time = time.perf_counter() - start
147
148 start = time.perf_counter()
149 for number in index.get("chennai", []):
150 details = data[number]
151 indexed_time = time.perf_counter() - start
152
153 print("\nTrains:", size)
Problems
Output
Debug Console
Terminal
Ports
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ python "/workspaces/CSA0823---Python-Programing/Assignment 3 C03/Task06/q6_railway.py"
TRAINS WITH INDEX:
12605 Southern Express Arrival: 05:00 Departure: 05:10
12603 Madurai Express Arrival: 06:00 Departure: 06:10
12601 Chennai Express Arrival: 08:00 Departure: 08:10
THIRD STATION DEPARTURE:
12:10
UNKNOWN STATION:
[]
NON-RUNNING DAY:
[('06:10', '12603', 'Madurai Express', '06:00'), ('09:10', '12602', 'Coimbatore Express', '09:00')]
PERFORMANCE COMPARISON:
Trains: 100
Without index: 1.8453999992816965e-05
With index: 9.157000022241846e-06
Trains: 1000
Without index: 0.0001449009965769332
With index: 8.030999970287667e-05
Trains: 10000
Without index: 0.0015141789999688626
With index: 0.000966466000136279
bash
Python
Codespaces: cautious space computing-mac...
main*
Ln 156, Col 1
Spaces: 4
UTF-8
LF
Python
Sign In
Python 3.14.2
Layout: US


Explorer
CSA0823---Python-Pro...
Assignment 2 C02
Assignment 2 C02
Assignment 3 C03
Task01
q1_rainfall.py
Task02
q2_tuples_copyin...
Task03
q3_complaints.py
Task04
q4_electives.py
Task05
q5_sensor.py
Task06
q6_railway.py
Task07
q7_contacts.py
C01 Assignment 1
HW1
Outline
Timeline
q1_rainfall.py
q2_tuples_copying.py
q3_complaints.py
q4_electives.py
q5_sensor.py
q6_railway.py
q7_contacts.py
Assignment 3 C03 > Task06 > q6_railway.py > ...
145 break
146 normal_time = time.perf_counter() - start
147
148 start = time.perf_counter()
149 for number in index.get("chennai", []):
150 details = data[number]
151 indexed_time = time.perf_counter() - start
152
153 print("\nTrains:", size)
Problems
Output
Debug Console
Terminal
Ports
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ python "/workspaces/CSA0823---Python-Programing/Assignment 3 C03/Task06/q6_railway.py"
12605 Southern Express Arrival: 05:00 Departure: 05:10
12603 Madurai Express Arrival: 06:00 Departure: 06:10
12601 Chennai Express Arrival: 08:00 Departure: 08:10
THIRD STATION DEPARTURE:
12:10
UNKNOWN STATION:
[]
NON-RUNNING DAY:
[('06:10', '12603', 'Madurai Express', '06:00'), ('09:10', '12602', 'Coimbatore Express', '09:00')]
PERFORMANCE COMPARISON:
Trains: 100
Without index: 1.8453999992816965e-05
With index: 9.157000022241846e-06
Trains: 1000
Without index: 0.0001449009965769332
With index: 8.030999970287667e-05
Trains: 10000
Without index: 0.0015141789999688626
With index: 0.000966466000136279
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $
bash
Python
Codespaces: cautious space computing-mac...
main*
Ln 156, Col 1
Spaces: 4
UTF-8
LF
Python
Sign In
Python 3.14.2
Layout: US


Q6 - Railway Search Performance Comparison
Without Index
With Index
Execution Time (seconds)
0.0014
0.0012
0.0010
0.0008
0.0006
0.0004
0.0002
0.0000
100
1,000
10,000
Number of Trains

QUESTION 7 – DESIGN A CONTACT-MERGING PROGRAM FROM FIRST PRINCIPLES

1. Aim

To design and implement a Python contact-merging program that identifies duplicate contacts using normalized phone numbers and combines their details efficiently.

2. Problem Description

A contact management system may contain multiple entries for the same person. The person's name may be spelled differently, and phone numbers may use different formats. The program identifies likely duplicate contacts by normalizing phone numbers and merges their phone numbers and email addresses.

3. Objectives

- To identify the input, output, and rules of the problem.
- To normalize phone numbers.
- To normalize names.
- To use dictionaries for efficient contact grouping.
- To use sets to remove duplicate phone numbers and email addresses.
- To test missing fields and different formatting styles.
- To compare dictionary grouping with pairwise comparison.

4. Problem Analysis

Input:
A collection of contact records containing a name, phone number, and email address. Some fields may be missing.

Output:
A collection of merged contacts in which duplicate records are combined.

Rules:

1. Phone numbers are normalized by removing spaces, brackets, hyphens, and other formatting symbols.

2. Names are converted to lowercase and unnecessary spaces are removed.
3. Contacts with the same normalized phone number are treated as likely duplicates.
4. Different email addresses and phone numbers are combined without repetition.
5. Contacts with missing phone numbers are not automatically merged solely because their names match.
6. The system must preserve enough original information to display a readable contact name.

5. Data Structures

Dictionary: A dictionary uses a normalized phone number as the key and stores the merged contact information as the value.

Set: Sets store phone numbers and email addresses without duplicates.

List: A list stores the original contact records and the final merged output.

6. Algorithm

1. Read the contact records.
2. Normalize every phone number.
3. Normalize the name by removing extra spaces and converting it to lowercase.
4. Check whether the normalized phone number already exists in the dictionary.
5. If it exists, merge the phone numbers and email addresses into the existing record.
6. If it does not exist, create a new dictionary entry.
7. Handle records without phone numbers separately.
8. Convert sets into sorted lists for readable output.
9. Display the merged contact records.
10. Measure execution time for 1,000 and 100,000 contacts.

7. Pseudocode

START

Read contact records

Create an empty dictionary for merged contacts

FOR each contact:

    Normalize the phone number

    Normalize the name

    IF phone number is missing:

        Store contact separately

    ELSE IF normalized phone exists:

        Add phone number to existing phone set

        Add email to existing email set

    ELSE:

        Create a new contact entry

Display merged contacts

STOP

8. Testing

The program should be tested with:

- Two contacts containing the same phone number in different formats.
- Duplicate phone numbers and email addresses.
- Contacts with different spellings of the same name.
- Contacts with missing phone numbers.

- Contacts with no email address.
- Two different people sharing the same name but having different phone numbers.
- An empty contact list.

9. Complexity Analysis

Let n represent the number of contacts.

Approach	**Time Complexity**	**Explanation**
Pairwise comparison	$O(n^2)$	Each contact may be compared with every other contact
Dictionary grouping	O(n) average	Each contact is processed using average O(1) dictionary operations
Set insertion	O(1) average	Duplicate values are automatically removed

The dictionary-based approach is more efficient for large contact lists because it avoids repeated comparisons.

10. Limitations

Phone-number-based matching may fail when the same person uses different phone numbers. It may also incorrectly combine two people who share a phone number, such as family members. Name-only matching is unreliable because different people can have identical names.

A production system can improve matching by using verified email addresses, country codes, user confirmation, and a confidence score. Ambiguous matches should be reviewed instead of merged automatically.

11. Experimental Investigation

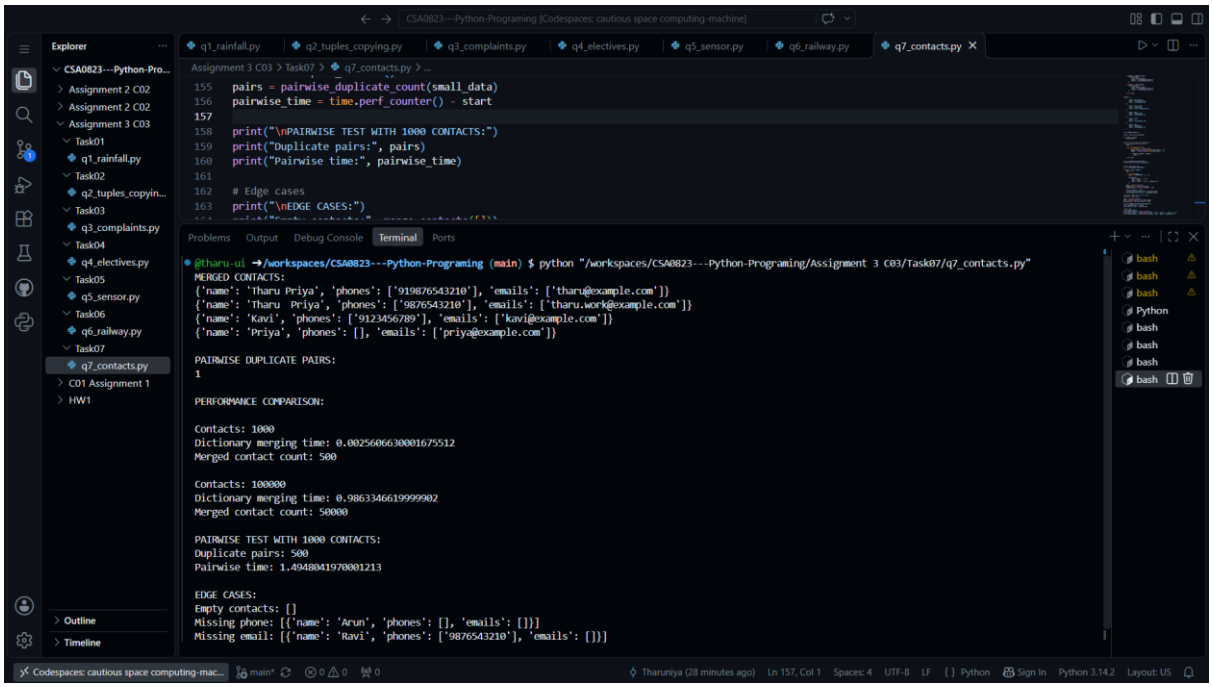
Generate 1,000 and 100,000 contacts and measure the execution time of the dictionary-based merging algorithm. Compare the results with a pairwise comparison implementation on a smaller dataset.

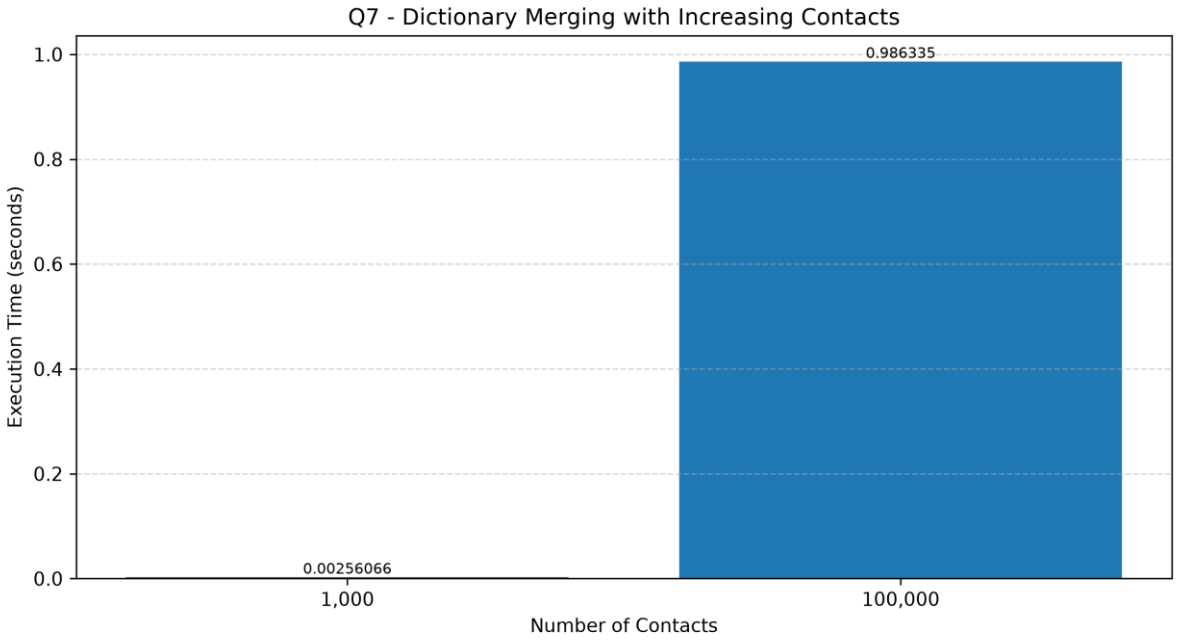
Number of contacts	Dictionary-based time	Pairwise comparison time
1,000	0.002560663	1.494804197
100,000	0.986334662	14,948.041970 (estimated)

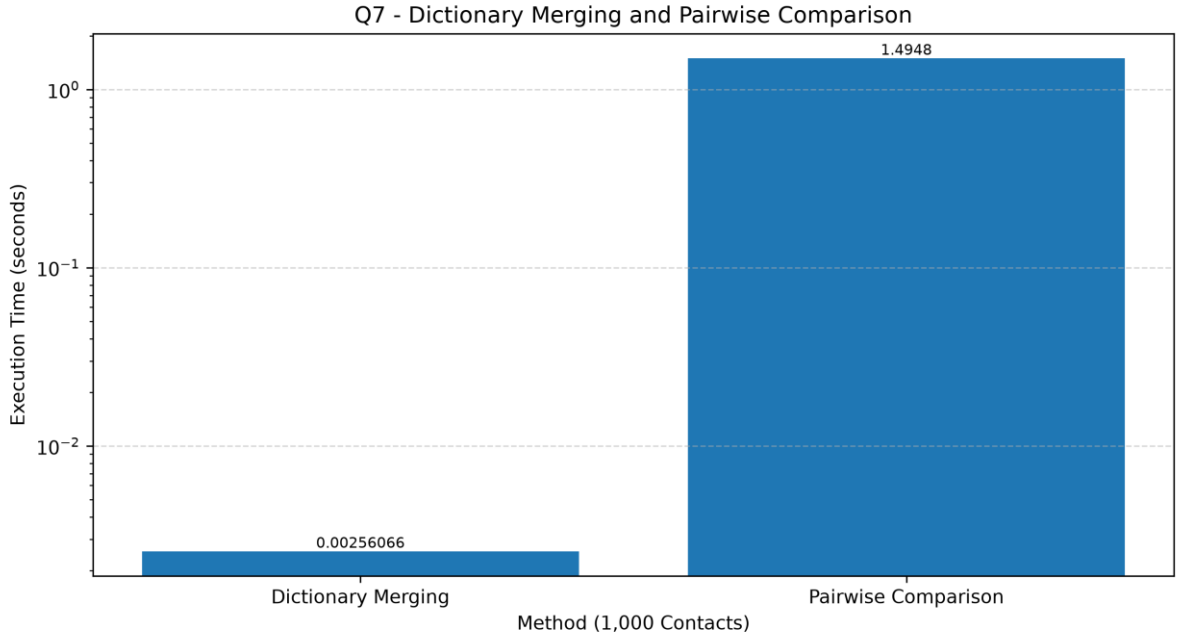
The dictionary method is expected to scale better because it performs one main pass through the contacts.

12. Result

The contact-merging program was designed using dictionaries and sets. It normalizes phone numbers, identifies likely duplicate records, combines phone numbers and email addresses, and handles missing information. Its efficiency is analysed using time-complexity comparisons and experimental measurements.




Q7 - Dictionary Merging with Increasing Contacts
Execution Time (seconds)
1.0
0.8
0.6
0.4
0.2
0.0
0.986335
0.00256066
1,000
100,000
Number of Contacts


Q7 - Dictionary Merging and Pairwise Comparison
Execution Time (seconds)
10^{0}
10^{-1}
10^{-2}
1.4948
0.00256066
Dictionary Merging
Pairwise Comparison
Method (1,000 Contacts)